Basic Python Syntax

Objectives

- Use correct Python syntax in Python programs.
- Use basic Python input and output functions properly.
- Write Python programs using the standard numeric data types and their operators.
- Use strings and their methods in Python programs.
- Convert between numeric and string data types.

Basic Syntax

This chapter deals with many of the issues relating to the fundamental syntax of the Python language.

Python is case sensitive. Therefore, the following two variables are different.

```
the_Person = "Him"
the_person = "Her"
```

Python statements do not end with a semicolon, but you can use one to separate two statements on the same line.

```
length = 10; width = 5
area = length * width
print(area)
```

All lines in a Python application must begin in the first column, unless the statement is in the body of a loop, conditional, function, or class definition.

- Python relies on indentation to determine the control flow structure of an application.
- As we introduce control structures, we will revisit the indentation rules.
- Line indentation is typically a multiple of four spaces.

You can continue a long statement by placing the \ character at the end of the line.

- When used in this fashion it is often referred to as the line continuation character.
- Several examples of its use are shown below.

```
message = "This is a long string just to \
illustrate continuation across multiple lines"

result = 500 * 2 + \
400 * 3
```

Statements continue onto the following line(s) when reaching the end of the line before the closing character for any of the following pairs of characters.

```
() [] {}
```

The following snippet of code uses the `print()` function with multiple arguments.

```
data = "This is just a string of text"
print("This is the first argument",
      "This is the second argument",
      data)
```

We indicate the arguments to the `print()` function using commas, and Python ignores the additional white spaces.

Although not recommended, Python allows any number of white space characters between the opening and closing parenthesis, which do not need to comply with indentation rules.

Simple Output

We have demonstrated the `print()` function in several forms in previous examples.

- This function takes zero or more arguments, separated by commas, and displays the data.
- The default behavior of the function is:
  - When there is more than one argument, `print()` displays each of the arguments separated by a single space.
  - `print()` displays a new line after the last parameter.

You can alter the default behavior by providing the following optional named arguments.

- `sep`
  - Separator character to display between arguments.
- `end`
  - Character to display after the last parameter.
- `file`
  - File to output the arguments.
  - Use of this named argument will be shown in a later chapter.

Although a more detailed explanation of named arguments will be discussed in a later chapter, the example below demonstrates their use with the `print()` function.

```
$ python3
Python 3.9.10 (main, Feb 9 2022, 00:00:00)
[GCC 11.2.1 20220127 (Red Hat 11.2.1-9)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello", 123, "Goodbye", 5 + 9, sep=":", end="###")
Hello:123:Goodbye:14###>>> exit()
$
```

Simple Input

The `input()` function provides a mechanism to obtain user input at runtime from the keyboard.

This function has several characteristics:

- Accepts an optional prompt string as an argument.
- Adds no spaces or other formatting characters.
- Automatically removes the trailing newline from the keyboard input.
- Always returns a string.

```
$ python3
Python 3.9.10 (main, Feb 9 2022, 00:00:00)
[GCC 11.2.1 20220127 (Red Hat 11.2.1-9)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> response = input("Please enter some text:")
Please enter some text:Hello
>>> response = input("Please enter some text:\n")
Please enter some text:
Goodbye
>>> print("The last response is", response, sep=":")
The last response is:Goodbye
>>> exit()
$
```

The following example shows how the function returns a string even when the user introduces numbers:

```
$ python3
Python 3.9.10 (main, Feb 9 2022, 00:00:00)
[GCC 11.2.1 20220127 (Red Hat 11.2.1-9)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> response = input("Please enter a number: ")
Please enter a number: 456
>>> print("The response of", response, "is of type", type(response))
The response of 456 is of type <class 'str'>
>>> exit()
$
```

Note
Converting the string to a number will be discussed later in this chapter.

Comments

Comments make source code more understandable and maintainable. Comments follow these general rules:

- Single-line comments begin with the # character and extend to the end of the physical line.
- An inline comment is a comment on the same line as a statement.
- PEP 8 suggests inline comments should be separated by at least two spaces from the statement.
- You can use triple-quoted strings (`"""`) to write multi-line comments.

The following code demonstrates a few examples of comments within the source code.

comments.py

```
#!/usr/bin/env python3
#
# This line and the single # character above are comments.
# This line is another comment.

number = 10                   # This is an inline comment
text = "hello there!"         # A simple greeting
data = "This # is not a comment "

# Sometimes comments are used to temporarily comment out
# sections of code as shown below. Although if the
# statements are actually not needed it would be better
# practice to remove the statements from the code entirely
# as opposed to commenting them out

# print(number)
# print(text)
# print(data)
```

Numbers

Python supports three distinct numeric data types.

- Integers, Floating Point Numbers, and Complex Numbers

Use the following table to verify the numerical operators precedence:

Numerical Operations

Operation	Result
`x + y`	Sum of x and y
`x - y`	Difference of x and y
`x * y`	Product of x and y
`x / y`	Quotient of x and y

Operation	Result
`x // y`	Floored quotient of x and y
`x % y`	Remainder of x/y
`-x`	x negated
`+x`	x unchanged
`abs(x)`	Absolute value (or magnitude) of x
`int(x)`	x converted to integer
`float(x)`	x converted to floating point
`divmod(x,y)`	The pair `(x // y, x % y)`
`pow(x, y)` or `x ** y`	x to the power y

Python can encode literal values in bases different than decimal. The following example shows some examples:

Base	Value	Decimal Value
Hexadecimal	`x = 0xff`	255
Octal	`y = 0o77`	63
Binary	`z = 0b111`	7

References
You can find more information about numeric data types in section 4.4 of the following URL:
`https://docs.python.org/3/library/stdtypes.html`

Strings

A string in Python is an immutable sequence of zero or more characters.

- Literal string values may be enclosed in single or double quotes.
- Literal strings can span multiple lines in several ways.
  - Using the line continuation character (`\`) as the last character.
  - Surrounding the string within a pair of matching triple-quotes: either double quotes(`"""`) or single quotes(`'''`).
- Literal strings may also be prefixed with a letter `r` or `R`.
  - These are referred to as raw strings and use different rules for backslash escape sequences.

The following is a list that represents all the escape sequences you can use in literal strings to represent special characters:

Escape Sequences

Sequence	Character/Meaning
`\newline`	Line continuation
`\\`	Backslash
`\'`	Single quote
`\"`	Double quote
`\a`	ASCII Bell (BEL)
`\b`	Backspace
`\f`	Form feed
`\n`	Linefeed
`\r`	Carriage Return
`\t`	Horizontal Tab
`\v`	Vertical Tab
`\ooo`	ASCII character (octal value `ooo`)
`\xhhh`	ASCII character (hex value `hhh`)
`\uxxxx`	Unicode Character with 16-bit hex value `xxxx`
`\Uxxxxxxxx`	Unicode Character with 32-bit hex value `xxxxxxxx`

The following example demonstrates the creation of literal strings by using the various techniques described in this chapter.

string_literals.py

```
#!/usr/bin/env python3
print("This is a literal string", 'and so is this')
print('"Double quotes" inside of single quotes')
print("'Single quotes' inside of double quotes")
print("A double quote \" inside double quotes")
print(r"A double quote \" inside a raw literal string")
print("A as unicode: \x41")

spades = """Royal Straight Flush \
\U0001F0A1 \U0001F0AE \U0001F0AD \U0001F0AB \U0001F0AA
"""
print(spades)

diamonds = """Royal Straight Flush \
```

```
\U0001F0C1 \U0001F0CE \U0001F0CD \U0001F0CB \U0001F0CA
"""
print(diamonds)
```

Running the above program produces the following output:

```
This is a literal string and so is this
"Double quotes" inside of single quotes
'Single quotes' inside of double quotes
A double quote " inside double quotes
A double quote \" inside a raw literal string
A as unicode: A
Royal Straight Flush 🂡🂮🂭🂫🂪

Royal Straight Flush 🃁🃎🃍🃋🃊
```

Figure 1.1: Output of the literal strings example

Note that while Python 3 fully supports Unicode characters, you still need a font supporting Unicode characters to produce this output.

String Methods

Python represents strings by using a class named `str`.

- If you are not familiar with Object Oriented Programming, you can think of a class as a new data type, and an object as an instance of that data type.
- Later in this course we will discuss Object Oriented Programming in more depth.

The `str` class has an abundance of methods defined within it.

- While some of the methods return Boolean values such as `True` or `False`, other methods return a modified version of the string on which they operate.

string_methods.py

```
#!/usr/bin/env python3
url = "www.redhat.com"
result = url.startswith("www")
print(result)
print(url.endswith(".org"))
new_url = url.upper()
print("ORIGINAL", url, "RETURNED", new_url)
```

When run, the preceding program outputs:

```
$ python3 string_methods.py
True
False
ORIGINAL www.redhat.com RETURNED WWW.REDHAT.COM
$
```

The following string methods return a Boolean indicating if the characters in the string are of a specific kind.

string_types.py

```
#!/usr/bin/env python3
print("AbCDe".isalpha(), "AbCd123".isalpha())   # True False
print("123".isnumeric(), "12.3".isnumeric())    # True False
print("  \t\n".isspace(), "a b\t\n".isspace())  # True False
print("ABCD".isupper(), "abcd".isupper())       # True False
```

The following snippet shows methods to search, replace, and split strings.

more_strings.py

```
#!/usr/bin/env python3

word = "is"
sentence = "The capital of Mississippi is Jackson."
position = sentence.find(word)
print("First:", position, "\t2nd: ", sentence.find(word, position + 1))
print(sentence.find(word, 8, 12))
print("The word '", word, "' appears", sentence.count(word), "times.\n")
print("Right Justified:", word.rjust(15), "|")
print(" Left Justified:", word.ljust(15, "*"), "|")

data = "1 4 1 1abc"
print("data:", data)
print("replace all:", data.replace("1", "0"))
print("replace two:", data.replace("1", "0", 2))

pieces = data.split(' ')
print(data)
print("pieces is of type:", type(pieces))
print(pieces)
```

The preceding programs outputs this:

```
$ python3 more_strings.py
First: 16  2nd:  19
-1
The word ' is ' appears 3 times.

Right Justified:              is |
 Left Justified: is************* |
data: 1 4 1 1abc
replace all: 0 4 0 0abc
replace two: 0 4 0 1abc
1 4 1 1abc
pieces is of type: <class 'list'>
['1', '4', '1', '1abc']
$
```

The following methods strip white space characters from a string.

whitespace.py

```
#!/usr/bin/env python3
# The following String has 3 sets of 2 spaces in it
data = "\t  \nabc  def\t  \n"

# The strip method removes leading and trailing whitespace
result = data.strip()
print(len(data), ":", len(result))

# The rstrip method removes trailing whitespace
result = data.rstrip()
print(len(data), ":", len(result))

# The lstrip method removes leading whitespace
result = data.lstrip()
print(len(data), ":", len(result))
```

The preceding code utilizes the built-in `len()` function to determine the length of each string and produces the following output:

```
$ python3 whitespace.py
16 : 8
16 : 12
16 : 12
$
```

References

You can find a complete list of string methods and their definitions in the Python documentation at the following URL:

https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str

Sequence Operations

Python strings are a sequence type (positionally ordered), and as such, share certain operations available to all types of sequences.

We will introduce additional sequence types, such as lists and tuples, in a later chapter.

The next example demonstrates the use of the following sequence operations with strings:

- The concatenation (`+`) operator.
- The repetition (`*`) operator.
- The membership testing (`in`) operator.

string_operations.py

```
#!/usr/bin/env python3
# The concatenation operator (+)
first_name = "Casey"
last_name = "Jackson"
full_name = first_name + " " + last_name
print(full_name)  # Casey Jackson

# Note the automatic string concatenation below
fullName = "Casey" " " "Jackson"
print(fullName)

# The asterisk (*) operator
stars = "*" * 12
pounds = 5 * "#"
print(stars, ":", pounds)  # ************ : #####

# The in operator is convenient for membership tests
x = "Hello there"
print('t' in x, 'ell' in x, 'hell' in x)  # True True False
```

Note that when you place literal strings next to each other, as seen on line 9, they are automatically concatenated even though there is no + operator between them.

Indexing and Slicing

Strings, like other sequence types, have a set of operations that utilize square brackets (`[]`) in the syntax.

All three of the following types of operations return a substring of the sequence the operation is performed on.

Indexing

Use one integer inside the brackets to get the element in that position, starting from zero. Negative values start the positional counting from the end of the sequence.

Slicing

Use two integers separated by a colon in the brackets to get the elements between the given positions.

Negative values also start from the end of the sequence, and omitted values mean `From the beggining` or `til the end` depending if you forgo the first or the second value.

Extended Slicing

Extended slicing behaves like regular slicing, but adds a third integer separated by a colon, which indicates the iteration step size to use when picking elements from the sequence. By default, the value of this parameter is one.

For example, `my_list[3:10:2]` starts from index 3 and stops at index `10`, selecting every two elements, because the step is 2.

The following example demonstrates these three operations:

indexes_and_slices.py

```
#!/usr/bin/env python3
spam = "Spam and eggs"
delim = " | "
# Indexing
print(spam[0], spam[3], spam[-1], spam[-4], sep=delim)

# Slicing
print(spam[2:7], spam[5:], spam[:8], sep=delim)

# Slicing from end
print(spam[-3:-1], spam[-3:], spam[:-1], sep=delim)
print()

# Extended Slicing
alphabet = "abcdefghijklmnopqrstuvwxyz"
print(alphabet[2:18:3])
start = 18
print(alphabet[start::1])
print(alphabet[::1])
```

Running the preceding program gives the following output:

```
$ python3 indexes_and_slices.py
S | m | s | e
am an | and eggs | Spam and
gg | ggs | Spam and egg

cfilor
stuvwxyz
abcdefghijklmnopqrstuvwxyz
$
```

Formatting Strings

Strings have a built in % operator that you can use to format stings similar to the `sprintf()` function in the C language.

- Using the % operator with strings may lead to a number of common errors.

References

You can find an in depth description of the % operator at the following URL:
https://docs.python.org/3/library/stdtypes.html#printf-style-string-formatting

- The newer `format()` method is the recommended way to avoid these errors and provide a more powerful way to format text.

The `format()` is a method of the `str` class that returns a copy of the string with each placeholder replaced with the value of the corresponding argument.

You can specify placeholders within the string by using the `{}` characters.

- If the placeholder characters surround an integer number, then it replaces the value by position.
- If the placeholder characters surround a word, then the substitution value is passed by keyword argument.

References

You can find a complete description of the various formatting options at the following URL:

`https://docs.python.org/3/library/string.html#formatstrings`

The following program demonstrates a few simple examples of using the `format()` method.

- We will introduce more complex examples later in the course.

format.py

```
#!/usr/bin/env python3
separator = "----------------------------"
name = "First: {} \tLast Name: {} \tMiddle Initial: {}"
formatted = name.format("John", "Smith", "C.")
print(formatted)
formatted = name.format("Melony", "Jones", "A.")
print(formatted)
print(separator)

name = "{1},  {0}"
print(name.format("First", "Last"))
print(name.format("John", "Smith"))
print(name.format("Melony", "Jones"))
print(separator)

dimensions = "Type: {type}\nHeight:{height}, Width:{width}"
result = dimensions.format(height=50, width=25, type="Box")
print(result)
```

The preceding program outputs:

```
$ python3 format.py
First: John   Last Name: Smith    Middle Initial: C.
First: Melony       Last Name: Jones     Middle Initial: A.
----------------------------
Last, First
Smith, John
Jones, Melony
----------------------------
```

```
Type: Box
Height:50, Width:25
$
```

Conversion Functions

When you need to treat a string as a numeric type, or a number as a string, you must convert the values.

Python provides several functions that perform these types of conversions.

The following table contains some of the most common built-in conversion functions:

`int()`	`str()`	`chr()`	`oct()`
`float()`	`ord()`	`hex()`	`bin()`

The `int()`, `float()`, and `str()` are constructors that initialize an object, as opposed to a function call.

We will discuss object initialization in more detail later in the course.

conversions.py

```
#!/usr/bin/env python3
# conversions to an integer
result = input("Please enter an integer: ")
number = int(result)
print("Your number plus 10 equals:", number + 10)
print("String of", result, "converted to various bases as int:")
fmt = "Base 10: {}\tBase 2: {}\tBase 8: {}\tBase 16: {}"
print(fmt.format(int(result), int(result, 2), int(result, 8), int(result, 16)))
print()
# conversions to a float
a_float = input("Please enter a decimal number: ")
sum_of_input = float(a_float) + float(result)
print(a_float, "+", result, "=", sum_of_input)
print()
print(number, "as string in the following bases:")
fmt = "Binary: {}\tOctal: {}\tHex: {}"
print(fmt.format(bin(number), oct(number), hex(number)))

print('ord("A") =', ord("A"), '     chr(66) =', chr(66))

print("The sum of the input equals", sum_of_input)
```

The preceding program outputs this:

```
$ python3 conversions.py
Please enter an integer: 101
Your number plus 10 equals: 111
String of 101 converted to various bases as int:
Base 10: 101  Base 2: 5
Base 8: 65    Base 16: 257
```

```
Please enter a decimal number: 23.45
23.45 + 101 = 124.45
101 as string in the following bases:
Binary: 0b1100101
 Octal: 0o145
   Hex: 0x65
ord("A") = 65      chr(66) = B
The sum of the input equals 124.45
$
```

If the arguments passed to the conversion functions do not match the expected format then Python raises an exception.

The following examples show the most common conversion errors:

```
$ python3
Python 3.9.10 (main, Feb 9 2022, 00:00:00)
[GCC 11.2.1 20220127 (Red Hat 11.2.1-9)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> result = int(input("Please enter a number: "))
Please enter a number: Hello
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'Hello'
>>> result = int("123", 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 2: '123'
>>> print(float("987"))     # While this works
987.0
>>> print(int(98.76))       # And this works also
98
>>> print(int("98.76"))     # This one does not work
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '98.76'
>>> exit()
$
```

Exercises

The solution files for these exercises are in the `/home/student/AD141-apps/basics/solutions` directory.

Exercise 1

Write a program that prompts to enter a string of text.

- The program should print the original text followed on a second line in the output by the number of characters entered.

Exercise 2

Write a program that prompts twice for text from the user.

- The first input should be a first name.
- The second input should be a last name.
- The program should print the full name on one line and the person's initials on the second line.

Exercise 3

Write a program that accepts a string from the user.

- Determine and print the following information about the string:
  - Does it end in a period?
  - Does it contain all alphabetic characters?
  - Is there an 'x' in the string?
- Create and print a new string that has all occurrences of e changed to E.

Exercise 4

Write a program that asks the user to enter a sentence.

- The program should determine and print the following information:
  - The first character in the string of text and the number of times it occurs in the string.
  - The last character in the string of text and the number of times it occurs in the string.

Exercise 5

Write an application that prompts to enter the radius of a circle.

- Accept the user input into a variable.
- Compute and print the area of the circle whose radius was input.
  - The formula for the area of a circle is πr^2 (pi times the square of the radius).
  - Use `3.14159` for pi.

Exercise 6

Write a program that prompts twice for an integer.

- Print the product of the two numbers.
- Once this works properly, try entering numbers with a decimal point.
  - What happens? Why?
- Now try entering data that is nonnumerical.
  - What happens? Why?

Exercise 7

Write a program that prompts the user for a string and then prompts again for a number.

- The program should create and print a new string by using the repetition operator on the string and the number.
  - For example, if the string is `hello` and the number is `3`, then `hellohellohello` should be printed.

Exercise 8

Write a program that prompts the user twice for a number.

- The first number will be the base, and the second number will be the exponent.
  - Print the result of raising the base to the exponent.

References

- You can find more information about numeric data types in section 4.4 of the following URL:
  `https://docs.python.org/3/library/stdtypes.html`
- You can find a complete list of string methods and their definitions in the Python documentation at the following URL:
  `https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str`
- You can find an in depth description of the % operator at the following URL:
  `https://docs.python.org/3/library/stdtypes.html#printf-style-string-formatting`
- You can find a complete description of the various formatting options at the following URL:
  `https://docs.python.org/3/library/string.html#formatstrings`

Summary

- Python is a high-level, interpreted, dynamically-typed programming language that relies on indentation to determine the control flow structure of your applications.
- The `input()` function obtains user input at runtime from the keyboard, and the `print()` function displays data to the user.
- Comments make your code more understandable and maintainable.
- Python supports the numeric and string data types that you can use, operate, and transform to fulfill the requirements of your application.